

Repurposing XQD Storage Paths for a Nikon Z7 and a Raspberry Pi 5

Executive summary

A Raspberry Pi 5 cannot realistically replace an XQD card inside a Nikon Z7 by wiring into the Pi's 22-pin display0 connector. The reason is foundational, not just inconvenient: the Pi 5's 22-pin CAM/DISP connectors carry MIPI DSI/CSI-2 signals for displays and cameras, while XQD is a PCIe-based storage card format. On the Pi 5, the only relevant high-speed storage-facing interface is the external PCIe FFC connector, which is a **host/root-complex** port intended to talk to peripherals such as NVMe drives; it is not documented as a PCIe endpoint/card-emulation interface. In other words, the Pi 5 can host storage devices, but it is not a practical way to impersonate the XQD/CFexpress card that the camera expects in its slot. ¹

The most realistic ways to get Z7 images onto a Pi are therefore **not slot emulation**. The best practical paths are: first, USB tethering the camera to the Pi and saving images onto Pi-attached storage; second, using an intact XQD reader as a USB ingest device after capture; third, if an official direct-to-computer workflow is preferred, using Nikon's documented computer/network workflows instead of trying to intervene on the card bus. Reusing a sacrificial XQD reader PCB may be useful only if it remains a complete USB reader; it is very unlikely to become a passive adapter from the camera's slot to the Pi's MIPI or GPIO pins. ²

The concise feasibility conclusion is therefore:

Goal	Feasibility	Bottom line
Pi 5 display0 22-pin port acting as an XQD card	Not feasible	Wrong protocol and wrong electrical interface. ³
Pi 5 PCIe port transparently emulating an XQD/CFexpress card in-slot	Extremely impractical	Wrong host/device role on Pi 5, unclear camera expectations, and significant signal-integrity and firmware work. ⁴
Classic USB tether from camera to Pi storage	Feasible	Best direct-to-Pi option. ⁵
Reader-attached ingest from card to Pi after capture	Feasible	Easiest hardware path, but not "write directly from camera to Pi." ⁶

What XQD publicly exposes

Public material from CompactFlash Association ⁷ describes XQD as a PCIe-based card format announced in 2012, with the same form factor as CFexpress Type B. The publicly visible CFA overview is conservative and lists XQD as PCIe Gen 1 with bus speeds up to 250 MB/s, while published card specifications from Sony

⁸ and Nikon ⁹ describe XQD Version 2-era cards/controllers using PCIe Gen 2 and USB 3.0/3.1-era controller features, with commercial cards around 440 MB/s read and 400 MB/s write. Taken together, the public picture is that XQD is definitely **PCIe-based and 3.3 V-powered**, but detailed version-by-version electrical behavior is less openly published than CFexpress. ¹⁰

A public connector datasheet from JST ¹¹ confirms that the same 21-circuit push-push socket family is designed for both CFexpress Type B and XQD cards. It specifies a 21-contact connector, 1.2 A rating on pin 10, 0.5 A on the other pins, and 12,000 mating cycles. A public CFexpress card datasheet from Delkin is especially useful because it shows the connector-level pin assignment and notes that **pin 7, reserved in CFexpress, is assigned to USBEN in the XQD specification**. That detail is important: even though XQD and CFexpress Type B share form factor and connector family, they are not trivially interchangeable at the protocol/control level. ¹²

The most relevant public pin groups are these:

XQD/CFexpress Type B card-edge group	Public signal assignment	Why it matters here
Pin 10	+3.3 V	The card interface is a 3.3 V storage interface, not a 5 V USB peripheral. ¹³
Pins 20/19 and 17/16	PCIe lane 0 TX/RX pairs	At least one PCIe lane is fundamental to the interface. ¹⁴
Pins 14/13	REFCLK+/REFCLK-	Storage timing comes from a PCIe reference clock, not MIPI clocking. ¹⁴
Pins 12/11/9	INS#, CLKREQ#, PERST#	Sideband detection, clock-request, and reset behavior must be satisfied. ¹⁴
Pins 6/5 and 3/2	Optional PCIe lane 1	Public CFexpress docs show optional second lane; older public XQD summaries imply lower-lane operation. ¹⁵
Pin 7	Reserved in CFexpress; USBEN in XQD note	A direct CFexpress-only assumption is unsafe if the body or reader still expects XQD semantics. ¹⁴

Another key distinction is protocol visibility. Public CFexpress documentation explicitly exposes NVMe command sets and NVMe-oriented behavior, but the public XQD material cited above describes XQD as a PCIe-based card format without making NVMe the center of the public story. That does **not** prove XQD is never NVMe-like in every implementation, but it does mean that “just make the Pi look like an NVMe SSD and the Z7 will treat it as XQD” is too strong an assumption for a risky hardware project. ¹⁶

What the Nikon Z7 publicly documents

Nikon’s current Z 7 / Z 6 manual states that the camera can be used with **XQD and CFexpress Type B** memory cards. The same documentation treats both as “memory cards” when no distinction is needed. Public Nikon documentation also shows the official computer workflows: connect the camera over USB and use NX Studio to copy pictures to a computer, or use a third-party card reader, or use network-oriented

workflows such as WT-7 plus Camera Control Pro 2/FTP-oriented features. In other words, Nikon’s public documents describe supported **storage media** and **computer/network transfer paths**, but they do not publish slot-level electrical or protocol behavior for the card interface. ¹⁷

That distinction matters. The public Nikon path for “save directly to computer” is not “make the card slot point somewhere else”; it is a separate tether/network workflow. Nikon’s manual explicitly says the camera can be controlled remotely and pictures saved directly to the computer when using Camera Control Pro 2 in the documented workflow, and the Z 7/Z 6 connection guide centers on USB transfer to NX Studio, not arbitrary storage redirection. For this project, that means an official direct-to-Pi architecture is much more likely to be a **tethering** workflow than a **card-slot proxy** workflow. ¹⁸

On the Linux side, the public libgphoto2 support matrix lists the Nikon Z7 as supporting image capture, trigger capture, live view, and configuration. That makes an unofficial Raspberry Pi workflow plausible over USB host mode, even though it is not the same as Nikon’s own Windows/macOS software stack. ¹⁹

Why the Pi 5 display0 port is the wrong connector

Official documentation from Raspberry Pi Ltd ²⁰ describes the Pi 5 as having **two mini 22-pin combined CSI/DSI ports**, a **single-lane PCIe interface**, two USB 3.0 host ports, two USB 2.0 host ports, a 40-pin GPIO header, and USB-C power input. Its product brief says earlier dedicated two-lane MIPI camera/display interfaces were replaced by a pair of **four-lane 1.5 Gbps MIPI transceivers**, while the external high-speed storage/peripheral expansion path is a **PCIe 2.0 x1 interface**. ²¹

The consequence is straightforward: the display0/CAM-DISP port is a MIPI D-PHY video connector, not a generic high-speed differential header. The official CM5IO high-speed schematic shows that the 22-pin CAM/DISP connectors are populated by D-PHY lane pairs, CAM/DISP GPIOs, I2C, 3.3 V, and ground. A reputable Raspberry Pi camera pinout reference shows the same family of 22-pin signals: interleaved grounds, MIPI data/clock pairs, enable/GPIO lines, I2C, and 3.3 V power. That is entirely different from the XQD/CFexpress card-edge expectations of PCIe TX/RX pairs, REFCLK, PERST#, CLKREQ#, and insertion-detect semantics. ²²

The Pi 5 PCIe connector, by contrast, *is* a storage-relevant interface. Official Raspberry Pi PCIe documentation shows a 16-pin, 0.5 mm FFC carrying 5 V, ground, PCIe Gen 2 x1 lane pairs, reference clock, reset, and two sideband GPIOs for power enable and detect/wake. Pins 1 and 2 provide 5 V, each rated 500 mA for a total of 1 A on the raw connector, while the official M.2 HAT+ converts that path to an M-key M.2 slot and supports up to 3 A to connected M.2 devices. This is the one Pi 5 interface that belongs in any serious high-speed storage design. ²³

For completeness, the relevant Pi 5 interfaces compare like this:

Pi 5 interface	Official electrical role	Useful for this project?	Core constraint
22-pin CAM/DISP 0 or 1	4-lane MIPI CSI/DSI transceiver port	No for XQD emulation	Wrong protocol family. ³

Pi 5 interface	Official electrical role	Useful for this project?	Core constraint
16-pin PCIe FFC	PCIe Gen 2 x1 host interface	Yes , but only for host-side storage/bridges	Pi is documented as host to peripherals, not as a card endpoint. ²⁴
USB 3.0 Type-A	Host ports for readers, drives, cameras	Yes	Best practical path for reader ingest or USB tethering. ²⁵
USB-C power/OTG	Power input; OTG peripheral mode only when board is powered elsewhere	Only marginally	Useful for Pi-as-gadget experiments, but not for talking to a camera that is itself a USB device. ²⁶
40-pin GPIO	3.3 V logic I/O	No for storage bus replacement	RP1 GPIO drive strengths are 2/4/8/12 mA; this is not a PCIe substitute. ²⁷

The PCIe FFC pinout published by Raspberry Pi is unusually helpful for adapter design:

Pi 5 PCIe FFC group	Signal set	Notes
Pins 1–2	5 V	500 mA each, 1 A total on the Pi connector. ²⁸
One differential pair	PCIE_TX_P/N	Host transmit pair. ²⁸
One differential pair	PCIE_RX_P/N	Host receive pair. ²⁸
One differential pair	PCIE_CLK_P/N	Reference clock. ²⁸
Sideband	PCIE_RST_B, PCIE_CLKREQ_N	Sideband signals are 3.3 V. ²⁹
Board-control GPIOs	PCIE_PWR_EN, PCIE_DET_WAKE	Needed by HAT+/adapter designs for proper power-up and bus probing. ³⁰

Power limits also matter. Raspberry Pi documentation says the Pi 5 provides up to **1.6 A total** to downstream USB peripherals when powered from a 5 A supply, but only **600 mA** with a 3 A supply. It also warns that disabling USB current limiting can cause instability, crashes, or data loss if peripherals overdraw the supply. That matters for card readers, bus-powered SSDs, and any bridge board fed from USB. ³¹

Architecture options and what each one buys you

The practical options are best understood as three different classes of system:

flowchart LR

```
A[Nikon Z7 card slot] --> B{Desired behavior}
B --> C[Transparent fake XQD/CFexpress card]
B --> D[USB tether to Raspberry Pi]
B --> E[Post-capture ingest via reader]

C --> C1[Needs PCIe endpoint/card emulation]
C1 --> C2[Custom FPGA/ASIC + interposer + firmware]
C2 --> C3[Very high risk / not recommended]

D --> D1[Camera USB-C to Pi USB 3 host]
D1 --> D2[gphoto2 or Nikon-style tether workflow]
D2 --> D3[Write to Pi SSD/NAS]

E --> E1[Real XQD card in camera]
E1 --> E2[Remove card]
E2 --> E3[XQD reader to Pi USB 3]
E3 --> E4[Checksum/copy to Pi storage]
```

The slot-level idea breaks down into two separate problems: electrical compatibility and role/protocol compatibility. Electrically, you would need a socket/interposer that preserves PCIe-quality routing, reference clocking, reset, insertion detect, and 3.3 V power sequencing at card-bus speeds. At the role level, the camera expects a **storage card endpoint**, whereas the Pi 5's public PCIe expansion story is that it **hosts** peripherals such as NVMe devices through M.2 adapters. Nothing in Raspberry Pi's official Pi 5 documentation describes the external PCIe connector as an endpoint-emulation feature. Combined with the fact that the public XQD story is not as openly specified as CFexpress/NVMe, this makes a transparent fake-card design a research-grade FPGA project, not a practical Pi hack. ³²

By contrast, intact-reader reuse is very plausible. Official Sony reader specs show host-side USB 3.x interfaces and support for XQD or XQD/CFexpress cards. That implies the reader PCB is an **active protocol bridge**, not passive pin-swapping. The right way to "repurpose" such a reader is to keep its controller and present it to the Pi over USB, where Raspberry Pi OS can handle it like other external storage devices. The wrong way is to imagine the reader PCB as a simple adapter to MIPI or GPIO. That would only work if the reader happened to expose a clean intermediate bus that matched what the Pi can host, which is not how commercial USB card readers are normally designed. This is an inference from the published reader capabilities, but it is the right engineering inference. ³³

The Pi's USB-C OTG mode is also a tempting dead end. Raspberry Pi's own OTG whitepaper says the Pi 5 can expose gadget mode on the USB-C power port **only when powered elsewhere**, e.g. through the 5 V/GND header, and that the recommended gadget modules include `g_mass_storage`, `g_ether`, and `g_serial`. That is useful if the *other* side is a USB host that knows how to talk to a mass-storage gadget. But a Nikon Z7 is publicly documented as a camera that connects to computers over USB for file transfer or tethering; it is not documented as a USB host that can mount arbitrary external mass-storage gadgets. So OTG gadget mode on the Pi 5 is technically interesting, but not a credible storage-substitution path for this camera. ³⁴

A comparison of the main approaches is below:

Approach	Hardware complexity	Software complexity	“Direct to Pi while shooting”	Risk to camera	Verdict
Pi 5 display0 wired toward XQD slot	Very high	Very high	Theoretically no	Very high	Reject immediately: wrong bus. ³
Pi 5 PCIe interposer trying to emulate XQD/CFexpress	Extreme	Extreme	Theoretically yes	Very high	Not practical without endpoint-capable custom hardware. ³²
Sacrificial reader reused as intact USB reader	Low	Low	No	Low	Good for ingest, not direct capture. ³⁵
USB tether from camera to Pi USB 3 host	Low	Moderate	Yes	Low	Best practical direct-to-Pi option. ⁵
Nikon network/PC workflow	Low	Moderate	Yes	Low	Officially clean, but less Pi-native. ¹⁸

Recommended implementation plans

Recommended plan for direct storage onto the Pi

Use the **camera’s USB port**, not the card slot.

1. Update Raspberry Pi OS and firmware. The official M.2 HAT+ guidance recommends `apt full-upgrade` and a current EEPROM; the same advice is sensible here because USB, PCIe, and storage behavior on Pi 5 improved over early firmware revisions. ³⁶
2. Attach persistent Pi-side storage, ideally NVMe through the official M.2 HAT+ or equivalent, because the Pi 5’s external PCIe path is the cleanest high-throughput local destination and Raspberry Pi OS automatically detects attached NVMe drives as `/dev/nvme0n1`. ³⁷
3. Connect the Z7 to a Pi USB 3 host port with a quality USB-C cable. Nikon’s public workflow is computer-over-USB, and libgphoto2 lists the Z7 as supported for image capture and control. ⁵
4. Run a tethered capture/import stack on the Pi. The conservative open stack is libgphoto2/gphoto2; if the use case is unattended ingest, wrap it in a systemd service or a small watcher that copies incoming files to the final directory structure and then to NAS/object storage. ¹⁹
5. If power draw is nontrivial, use the official 27 W supply or a powered USB 3 hub. Raspberry Pi’s documentation is explicit that peripheral power headroom depends strongly on the PSU. ³⁸

This plan is the only one here that satisfies the spirit of “images land on the Pi during shooting” without exposing the Z7 card bus to experimental hardware. ⁵

Recommended plan for using the sacrificial XQD reader

Use the reader as a **USB ingest bridge**, not as a raw slot adapter.

1. Keep the reader electronics intact. Official Sony reader specs show a USB 3.x interface and bus-powered design, which means the controller is doing real protocol conversion. ³⁵
2. Connect the reader to a Pi USB 3 port or to a powered USB 3 hub if the overall setup also includes SSDs or other high-power devices. ³⁹
3. Configure the Pi to auto-mount or auto-import the card contents to SSD/NVMe. Raspberry Pi OS already has the required host-side USB stack; in most cases the only question is whether the reader binds cleanly with UAS or falls back to bulk-only storage. The Pi 5 product brief specifically calls out improved throughput to external UAS drives, and Linux USB documentation explains why UAS matters for queue depth and command concurrency. ⁴⁰
4. Treat this as a safe ingest station, not as direct camera storage.

This is the best use of a reader you are willing to sacrifice, and it materially improves workflow, but it does **not** make the Pi a fake card. ³⁵

Experimental plan for research only

A true in-slot proxy or emulator would require all of the following:

- an XQD/CFexpress Type B socket or interposer with controlled-impedance routing;
- a genuine PCIe endpoint-capable device, likely an FPGA or custom ASIC, not a stock Pi 5 external connector role;
- firmware implementing card-side behavior the camera accepts, including reset, detect, power sequencing, and likely storage protocol semantics beyond simple “there is a PCIe link”;
- a safe 3.3 V power design and hot-plug discipline;
- high-speed validation equipment. ⁴¹

That is a serious hardware program, not a reasonable hack unless the objective is reverse engineering itself.

Software, power, performance, and risk

The Raspberry Pi OS software pieces differ by architecture:

Use case	Likely kernel/user-space components	Notes
NVMe storage on Pi PCIe	<code>nvme</code> , <code>nvme_core</code> , current Pi EEPROM/firmware	Official Pi docs show automatic detection of NVMe storage and recommend current firmware. ⁴²
USB reader ingest	xHCI host stack, <code>uas</code> or <code>usb-storage</code> , normal block/filesystem tools	Pi 5 docs emphasize higher USB/UAS performance to external drives. ⁴³

Use case	Likely kernel/user-space components	Notes
Pi OTG gadget mode	<code>dwc2</code> , <code>g_mass_storage</code> or <code>ConfigFS/usb_gadget</code>	Officially possible on Pi 5 USB-C only when powered elsewhere, but not useful for a camera that is itself USB-device-oriented. ⁴⁴
USB camera tethering	<code>libgphoto2/gphoto2</code> and optionally a wrapper service	Public camera matrix lists Nikon Z7 support. ¹⁹

For performance, the official Pi-side ceiling is clear: the Pi 5 external PCIe path is PCIe 2.0 x1 with a 500 MB/s peak transfer rate on the official M.2 HAT+ path. Reader-side published USB 3.x readers are therefore often the limiter anyway. Independent reader testing found Sony G-series XQD cards hitting about 344 MB/s read and roughly 246–255 MB/s write through common XQD readers, i.e. comfortably below exotic custom-card-bus ambitions but fast enough for ingest workflows. This is another reason to prefer a real reader or a tethered workflow over a fake-card design. ⁴⁵

Power and hot-plugging deserve special caution. The public Delkin CFexpress electrical spec gives a card operating voltage window of roughly 3.14–3.47 V and a minimum off-time of 1 second before power is reapplied. Raspberry Pi's PCIe FFC exposes 5 V and expects add-on boards to manage local regulation and power sequencing; the official M.2 HAT+ does exactly that. Any attempt to wire the camera slot directly to improvised hardware would therefore have to solve 3.3 V regulation, sideband timing, and surprise-removal behavior correctly. Done badly, this is an easy path to card corruption or camera damage. ⁴⁶

The legal and warranty side is also nontrivial. CFA's public XQD page states that the XQD trademark and logo are owned solely by Sony and licensed through CFA. That does not stop personal experimentation, but it does mean that any plan to publish, sell, or badge a device as "XQD-compatible" wanders into conformance, licensing, and trademark territory very quickly. Separately, opening the reader or building a card-slot interposer will almost certainly void warranties and may compromise reliability even if it "works once." ⁴⁷

A small parts list for the two realistic plans looks like this:

Item	Needed for	Current rough price signal	Notes
Official Raspberry Pi M.2 HAT+	Pi-local NVMe storage	Available from \$12. ⁴⁸	Good add-on if the Pi is the destination during tethered shooting.
Official Raspberry Pi USB 3 Hub	Extra powered USB headroom	Official launch pricing \$12. ⁴⁹	Useful if reader + SSD + peripherals exceed comfortable bus power.
Official Raspberry Pi 27 W USB-C PSU	Reliable peripheral power	Official product page positions it for Pi 5 plus SSDs/high-power USB devices. ⁵⁰	Strongly recommended if storage and USB devices are attached.

Item	Needed for	Current rough price signal	Notes
Sony MRW-E90 or similar XQD reader	Reader-ingest plan	Representative listings are around \$50-\$80 used/new-old-stock. ⁵¹	You already own one, so incremental cost may be zero.
Existing XQD card and Nikon Z7	Both realistic plans	Already owned	No reason to sacrifice the card for the recommended paths.

The risk-mitigation guidance is therefore simple. Do not attach anything to the Pi 5 display0/CAM-DISP connector for this purpose. Do not hot-plug experimental interposers into the camera card slot. Use the official Pi 5 PCIe path only for *Pi-hosted* storage, not camera-slot emulation. Prefer USB tethering if “direct to Pi while shooting” is the real goal. Prefer reader ingest if robustness matters more than immediacy. If research into slot-level emulation continues, isolate it from the camera first with a standalone host fixture, not the Z7 body itself. ⁵²

flowchart TD

A[Best practical recommendation] --> B{Need images on Pi during capture?}

B -->|Yes| C[USB tether Nikon Z7 to Pi USB 3]

C --> D[libgphoto2 or equivalent ingest daemon]

D --> E[Write to Pi NVMe / NAS / object store]

B -->|No, ingest after shoot is fine| F[Use intact XQD reader over USB 3]

F --> G[Auto-import to Pi storage]

A --> H[Do not use display0/CAM-DISP for XQD]

A --> I[Do not attempt fake-card emulation unless reverse engineering is the project]

Final recommendation

If the actual objective is “**my Z7’s images end up on the Raspberry Pi while I shoot**”, the recommended path is **USB tethering from camera to Pi**, with Pi-local SSD/NVMe or network storage as the sink. If the objective is “**fast, Pi-based storage workflow using my existing XQD media**”, the recommended path is **reader-based ingest over USB 3**. Both are realistic, comparatively safe, and supported by the public behavior of the camera and the Pi. ⁵³

If the objective is specifically “**make the Pi look like an XQD card in the Z7 slot**”, the answer is that this is not a sane Raspberry Pi 5 display0 project and is barely a sane Pi 5 PCIe project. It is a custom high-speed endpoint-emulation project with unclear XQD-specific behavior, substantial mechanical and electrical risk, and little reason to prefer it over the cleaner USB tether path. ⁵⁴

¹ ⁸ ²⁵ ⁴⁰ ⁴³ <https://pip.raspberrypi.com/documents/RP-008348-DS-raspberry-pi-5-product-brief.pdf>
<https://pip.raspberrypi.com/documents/RP-008348-DS-raspberry-pi-5-product-brief.pdf>

2 5 53 https://onlinemanual.nikonimglib.com/z7_z6/en/10_connections_02.html
https://onlinemanual.nikonimglib.com/z7_z6/en/10_connections_02.html

3 21 31 38 52 54 <https://www.raspberrypi.com/documentation/computers/raspberry-pi.html>
<https://www.raspberrypi.com/documentation/computers/raspberry-pi.html>

4 23 24 28 29 30 32 <https://pip.raspberrypi.com/documents/RP-008298-DS-pcie-connector-standard.pdf>
<https://pip.raspberrypi.com/documents/RP-008298-DS-pcie-connector-standard.pdf>

6 33 35 <https://www.sony.com/electronics/support/accessories/mrw-e90/specifications>
<https://www.sony.com/electronics/support/accessories/mrw-e90/specifications>

7 20 51 <https://focalpointphoto.com/shop/sony-mrw-e90-xqd-sd-card-reader/3ea05030-7375-0134-bae1-00163e9110c0?variation=3876689>
<https://focalpointphoto.com/shop/sony-mrw-e90-xqd-sd-card-reader/3ea05030-7375-0134-bae1-00163e9110c0?variation=3876689>

9 10 47 <https://compactflash.org/card-types/>
<https://compactflash.org/card-types/>

11 13 14 15 16 41 46 <https://www.farnell.com/datasheets/2815914.pdf>
<https://www.farnell.com/datasheets/2815914.pdf>

12 <https://www.jst.co.uk/downloads/series/eXCC%2819-08-08%29.pdf>
<https://www.jst.co.uk/downloads/series/eXCC%2819-08-08%29.pdf>

17 https://download.nikonimglib.com/archive5/ciY3y00Leaei05IwZML61uw0Iz21/Z7Z6UMNSG_NSNGTH%28En%2907.pdf
https://download.nikonimglib.com/archive5/ciY3y00Leaei05IwZML61uw0Iz21/Z7Z6UMNSG_NSNGTH%28En%2907.pdf

18 https://onlinemanual.nikonimglib.com/z7_z6/en-nsg/14_technical_notes_03.html
https://onlinemanual.nikonimglib.com/z7_z6/en-nsg/14_technical_notes_03.html

19 <https://www.gphoto.org/proj/libgphoto2/support.php>
<https://www.gphoto.org/proj/libgphoto2/support.php>

22 <https://pip.raspberrypi.com/documents/RP-008182-DS-cm5io-datasheet.pdf>
<https://pip.raspberrypi.com/documents/RP-008182-DS-cm5io-datasheet.pdf>

26 34 44 <https://pip.raspberrypi.com/categories/685-app-notes-guides-whitepapers/documents/RP-009276-WP/Using-OTG-mode-on-Raspberry-Pi-SBCs.pdf>
<https://pip.raspberrypi.com/categories/685-app-notes-guides-whitepapers/documents/RP-009276-WP/Using-OTG-mode-on-Raspberry-Pi-SBCs.pdf>

27 <https://pip.raspberrypi.com/documents/RP-008370-DS-1-rp1-peripherals.pdf>
<https://pip.raspberrypi.com/documents/RP-008370-DS-1-rp1-peripherals.pdf>

36 37 42 45 <https://www.raspberrypi.com/documentation/accessories/m2-hat-plus.html>
<https://www.raspberrypi.com/documentation/accessories/m2-hat-plus.html>

39 <https://www.raspberrypi.com/documentation/computers/configuration.html>
<https://www.raspberrypi.com/documentation/computers/configuration.html>

48 <https://www.raspberrypi.com/products/m2-hat-plus/>
<https://www.raspberrypi.com/products/m2-hat-plus/>

49 <https://www.raspberrypi.com/news/raspberry-pi-usb-3-hub-on-sale-now-at-12/>
<https://www.raspberrypi.com/news/raspberry-pi-usb-3-hub-on-sale-now-at-12/>

50 <https://www.raspberrypi.com/products/27w-power-supply/>
<https://www.raspberrypi.com/products/27w-power-supply/>